

多模态学习笔记

导航

1 前言	3
2 准备知识	3
2.1 大模型训练	3
2.1.1 DeepSpeed	3
2.1.2 FullyShardedDataParallel	4
2.1.3 ColossalAI	4
3 总览图	4
4 CLIP	5
4.1 Prompt engineering and ensembling	6
4.2 特征学习	6
4.3 CLIP 的局限性	6
5 LSeg	7
5.1 模型结构	8
5.2 Spatial regularization	8
5.3 局限性	8
6 GroupViT	8
6.1 模型结构	9
6.2 Grouping Block	10
6.3 模型推理	10
6.4 局限性	10
7 ViLD	11
7.1 模型结构	11
7.2 训练阶段	12
8 GLIP	12

9 GLIPv213

10 ALBEF13

10.1 模型结构 13

10.2 Image-Text Contrastive Learning, ITC 14

10.3 Masked Language Modeling, MLM 14

10.4 Image-Text Matching, ITM 14

11 BLIP15

12 CoCa15

13 BeiTv315

14 SAM15

14.1 image encoder 15

14.2 prompt encoder 15

14.3 mask decoder 16

15 YOLACT17

15.1 训练过程 17

15.2 推理过程 18

16 FastSAM18

16.1 全实例分割 18

16.2 目标匹配 18

17 Grounding DINO18

17.1 Feature Extraction and Enhancer 19

1 前言

以下是在学习基于深度学习方法实现多模态的过程中，总结的论文、学习报告、博客等等，以及针对部分问题提出的个人看法，特此记录！

2 准备知识

2.1 大模型训练

在训练大模型时，需要庞大的资源。单卡显存通常不能满足训练的需要，因此出现了大模型训练技术，其中典型的一种是 DeepSpeed ZeRO

2.1.1 DeepSpeed

DeepSpeed 是微软开源的基于 PyTorch 的分布式框架，其支持了 ZeRO, 3D-Parallelism, DeepSpeed-MoE, ZeRO-Infinity 等训练策略。其中，DeepSpeed ZeRO 支持切分优化器、梯度以及参数。

需配置 FlexibleRunner 的 strategy 和 optim_wrapper 参数：

- strategy: 指定 type='DeepSpeedStrategy' 并配置参数
- optim_wrapper: 指定 type='DeepSpeedOptimWrapper' 并配置参数

```
from mmengine.runner._flexible_runner import FlexibleRunner
```

```
# 指定 DeepSpeedStrategy 并配置参数
```

```
strategy = dict(
    type='DeepSpeedStrategy',
    fp16=dict(
        enabled=True,
        fp16_master_weights_and_grads=False,
        loss_scale=0,
        loss_scale_window=500,
        hysteresis=2,
        min_loss_scale=1,
        initial_scale_power=15,
    ),
    inputs_to_half=[0],
    zero_optimization=dict(
        stage=3,
        allgather_partitions=True,
        reduce_scatter=True,
        allgather_bucket_size=50000000,
        reduce_bucket_size=50000000,
        overlap_comm=True,
        contiguous_gradients=True,
        cpu_offload=False),
)
```

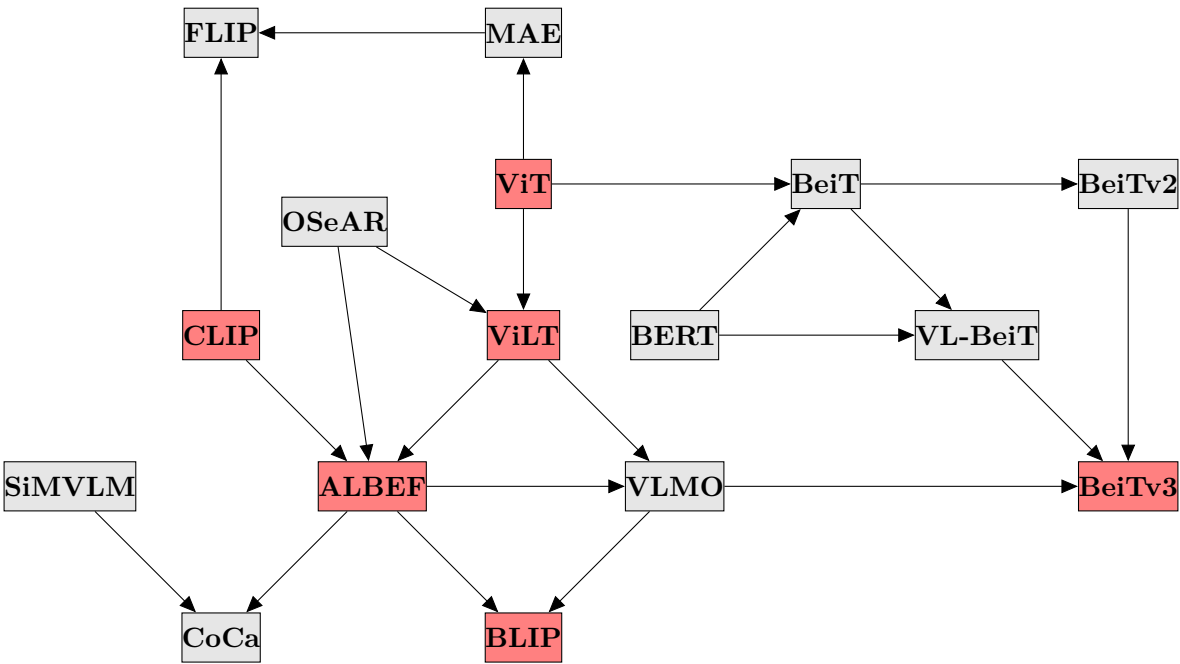
```
# 指定 DeepSpeedOptimWrapper 并配置参数
```

```
optim_wrapper = dict(  
    type='DeepSpeedOptimWrapper',  
    optimizer=dict(type='AdamW', lr=1e-3))  
  
# 初始化 FlexibleRunner  
runner = FlexibleRunner(  
    model=MMResNet50(),  
    work_dir='./work_dirs',  
    strategy=strategy,  
    train_dataloader=train_dataloader,  
    optim_wrapper=optim_wrapper,  
    param_scheduler=dict(type='LinearLR'),  
    train_cfg=dict(by_epoch=True, max_epochs=10, val_interval=1),  
    val_dataloader=val_dataloader,  
    val_cfg=dict(),  
    val_evaluator=dict(type=Accuracy))  
  
# 开始训练  
runner.train()
```

2.1.2 FullyShardedDataParallel

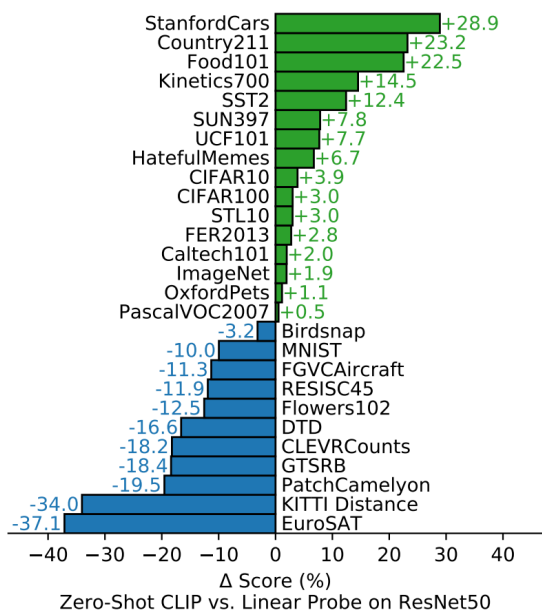
2.1.3 ColossalAI

3 总览图

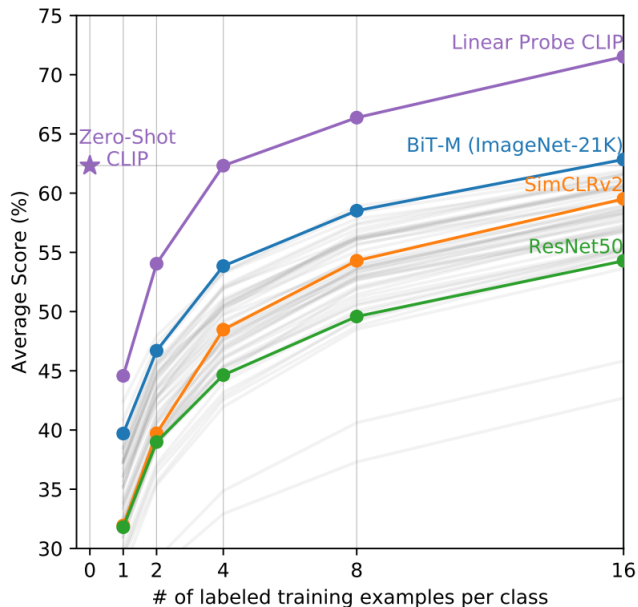


4 CLIP

迁移学习，泛化性强



(a) zoo-shot



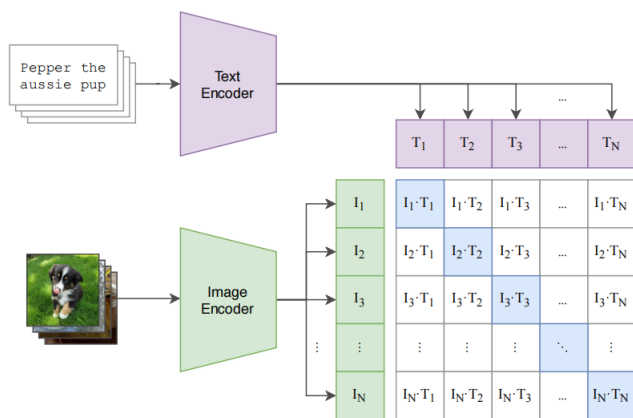
(b) few-shot

提醒： CLIP 为什么能成功？

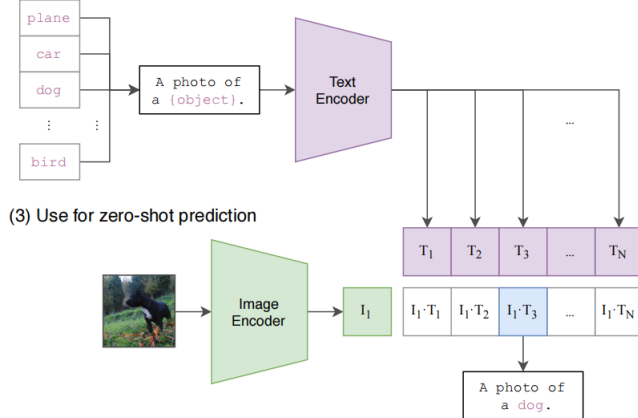
以前目标类别有限，且数据量不够大；模型不够大；算力资源有限制。

训练的时候是通过文本和图片**对比学习**【图片和文本描述对应上的是正样本，其他的为负样本，二分类】，测试的时候，通过计算图片编码特征和文本编码特征计算相似度，实现 zoo-shot

(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction

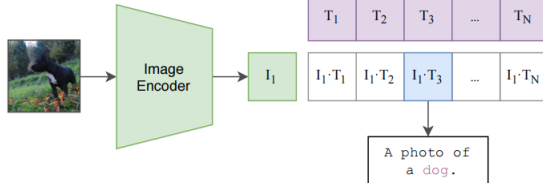


Figure 1. Summary of our approach. While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset's classes.

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T) #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2
```

4.1 Prompt engineering and ensembling

[prompt template](#) 【缩小提示空间、强调】，解决文本歧义问题

4.2 特征学习

衡量模型学到的特征好不好，有两种方式：

- Linear probe：把预训练好了的参数冻住，然后再训练一个分类头 【可学习的空间较小】
- Fine tune：开放网络参数，直接端到端学习 【往往比 Linear probe 的效果要好】

提醒： CLIP 为什么使用 Linear probe？

主要是因为：

- CLIP 的目的就是为了研究和数据集无关的与训练方式，如果下游任务的数据集足够大，且开放网络参数微调的话，很有可能是预训练的模型并不好，但是微调之后效果很好，**无法判断预训练参数到底好不好**
- 下游任务不需要调参，不需要为不同的数据集设计超参数，流程正规

4.3 CLIP 的局限性

- CLIP 很强，但是没有强到不可一世。如果加大数据和模型规模，CLIP 的性能还能继续提高，但是计算量大概还需要 $\times 1000$
- 细分类数据集效果不好，且无法处理一些抽象的任务
- out of distribution，CLIP 也不好，比如 MNIST

- 还是需要给定类别
- 数据用量太大，可以考虑自监督或伪标签
- 因为测试集是精心挑选的，无形带入了偏见
- 语言描述本身存在局限性

5 LSeg

有监督训练

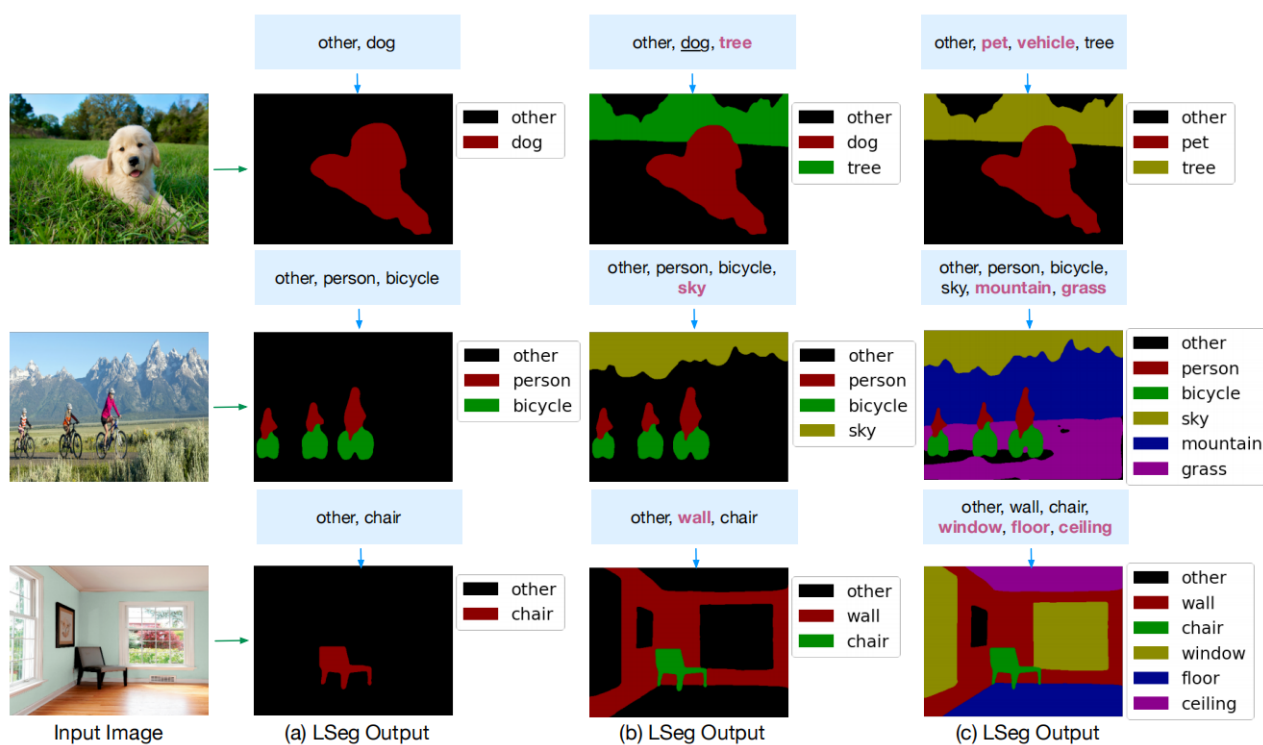
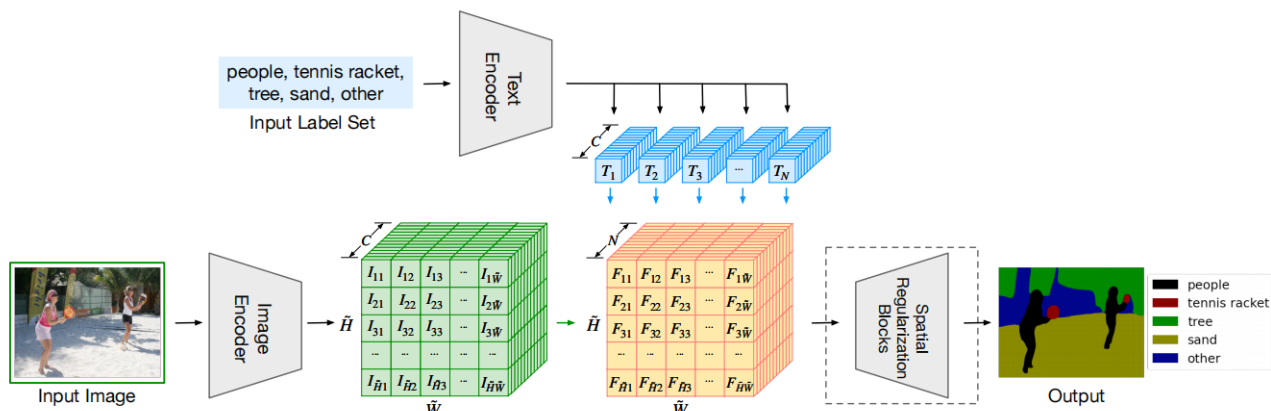


图 2: 示例结果。LSeg 能够处理看不见的标签以及任意长度和顺序的标签集

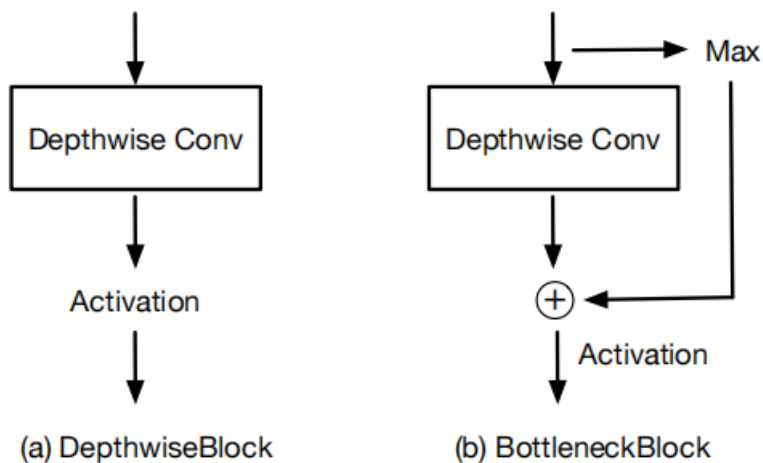
5.1 模型结构



至始至终文本编码器都是锁住的【数据集较小，fine-tune 会把模型带偏了】

5.2 Spatial regularization

空间正则化的目的是在不破坏像素之间标签关系的前提下,将预测的嵌入映射回原始图像的空间维度。【由于内存限制，图像编码器通常在低于输入图像分辨率的较低分辨率下预测像素嵌入】



5.3 局限性

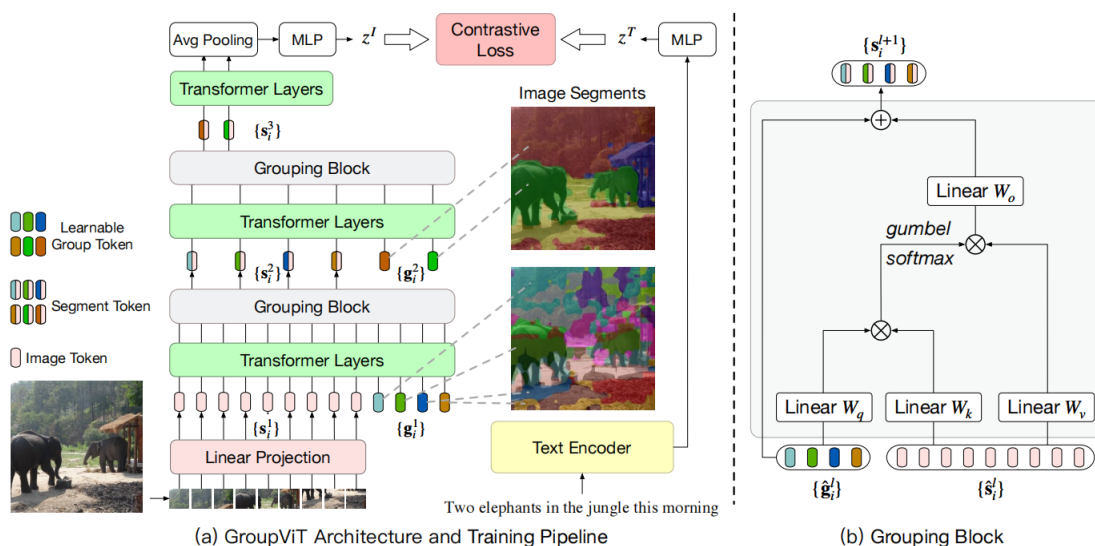
- 还是依赖于手工标注
- 不是在做真正的分类，只是在按概率分配

6 GroupViT

不依赖于手工标注，仅需要图像-文本对



Figure 7. Concepts Learnt by Group Tokens.. We highlight the regions that group tokens attend to in different stages.



6.1 模型结构

主要流程为：

- 利用 ViT 将原始图片编码为 196×384 的特征，并初始化 group 为 64×384 【64 可以视为初始化聚类中心，384 是为了和图片特征对齐】
- 经过 Grouping Block 完成聚类中心分配 【聚类可视化发现模型确实拥有这样的功能】
- 后一个 Transformer Layers，初始化聚类中心个数为 8
- Avg Pooling 【为了与文本特征维度对齐】

6.2 Grouping Block

使用 Gumbel-Softmax 计算 Group Tokens 和图像段 Tokens 之间的相似性。每个图像段 Token 被分配给与之最相似的一个 Group Token。这个过程通过 one-hot 操作实现【不是可微分的】，确保每个图像段 Token 只属于一个 Group。

6.3 模型推理

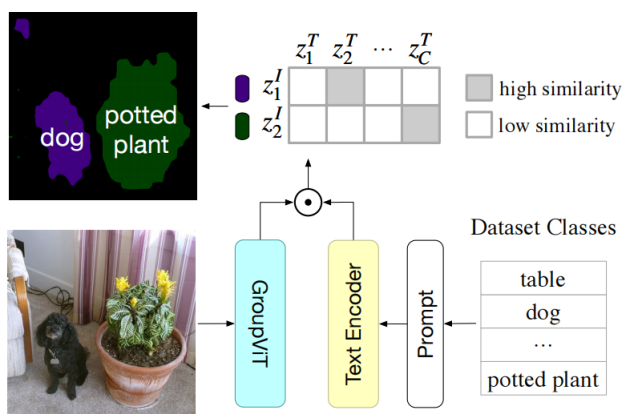


Figure 4. **Zero-Shot Transfer of GroupViT to Semantic Segmentation.** Each output segment's embedding from GroupViT corresponds to a region of the image. We assign each output segment to the object class with the highest image-text similarity in the embedding space.

推理流程为：

- ViTd 得到 8 个 Group Embedding
- 有可能的标签经过文本编码器得到文本特征
- 计算相似度，得到每个 Group Embedding 对应的 class

6.4 局限性

- 与有监督的方法差距比较大
- 最多只能分 8 类

arch	# group tokens	# output tokens	mask mIoU
GroupViT	16	4	28.6
GroupViT	16	8	37.1
GroupViT	32	8	38.3
GroupViT	64	8	39.3
GroupViT	64	16	38.0

Table 2. Ablation results of different numbers of group and output tokens.

- 背景类干扰【目标类别过多，导致前景置信度较低，过滤阈值不好把控】

7 ViLD

在 ViLD 之前, $VE > TE > MI \Rightarrow VE = TE > MI \Rightarrow VE > MI > TE$, ViLD 得出的结论是 $MI > VE = TE$

7.1 模型结构

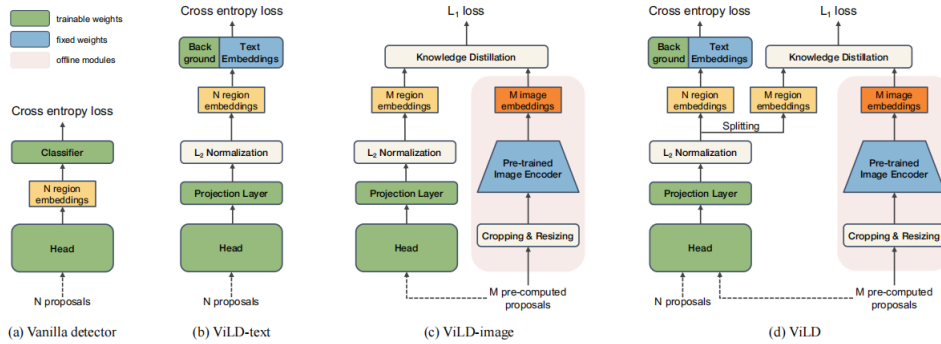
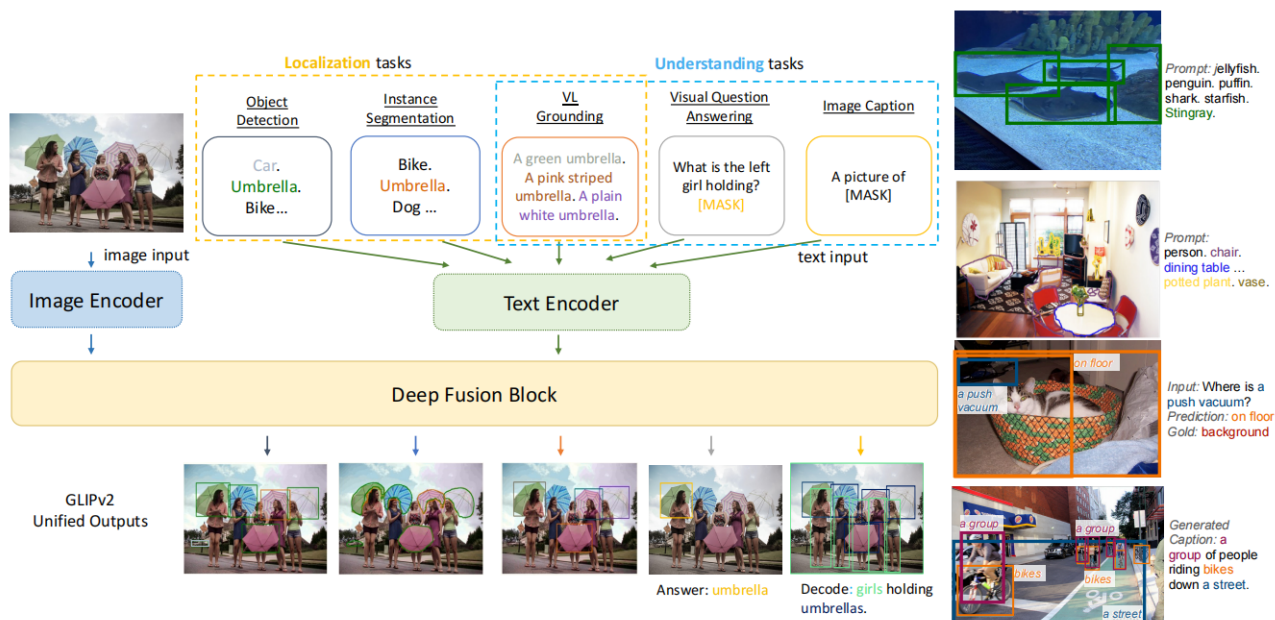


Figure 3: **Model architecture and training objectives.** (a) The classification head of a vanilla two-stage detector, *e.g.*, Mask R-CNN. (b) ViLD-text replaces the classifier with fixed text embeddings and a learnable background embedding. The projection layer is introduced to adjust the dimension of region embeddings to be compatible with the text embeddings. (c) ViLD-image distills from the precomputed image embeddings of proposals with an $\mathcal{L}_1$ loss. (d) ViLD combines ViLD-text and ViLD-image.

9 GLIPv2



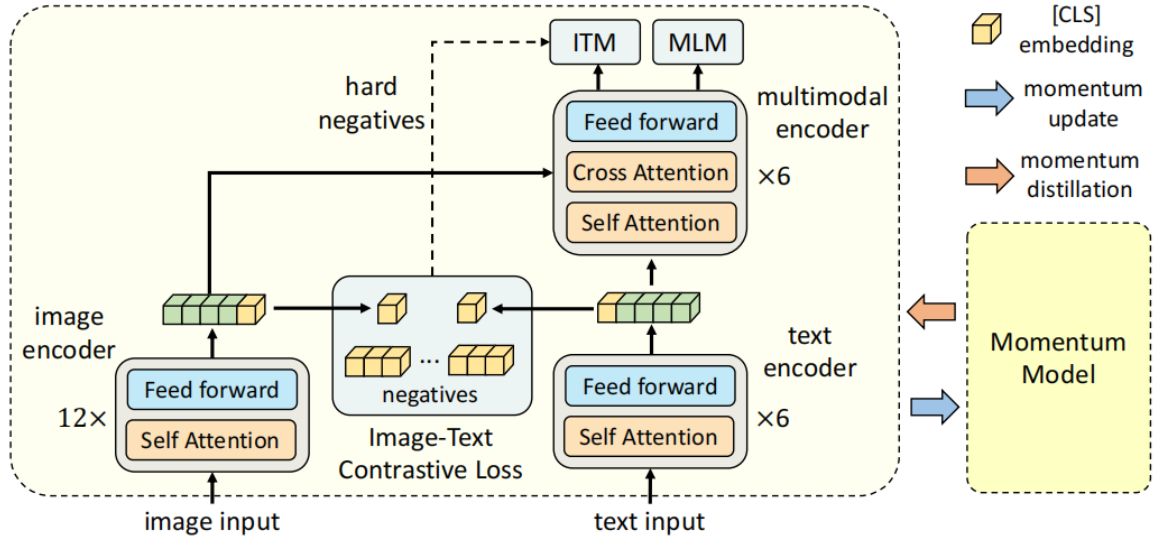
10 ALBEF

在此之前相关研究存在的问题：

- 用于抽取图像特征和文本特征的模型是分别单独训练的【特征不在一个空间内】
- 使用目标检测模型作为图像特征的抽取器很“贵”，在预训练阶段，需要标注物体的 box 位置，在预测阶段需要有高分辨率的图片
- 目前广泛使用的图像-文本数据集存在着固有的噪声，这些噪声会影响模型性能

10.1 模型结构

整体结构与7中的 c 类结构比较接近



10.2 Image-Text Contrastive Learning, ITC

只涉及图像和文本的 encoder，主要目的是通过该任务使得图像编码器的向量空间与文本编码器的向量空间对齐【通过计算两个特征之间的相似度实现】

针对数据噪声问题，在 ITC 任务中使用了 Momentum Distillation 【训练的过程中将伪标签加入 GT】。

10.3 Masked Language Modeling, MLM

完形填空

10.4 Image-Text Matching, ITM

ITM 和 ITC^{10.2}很相似，区别在于 ITC 只通过 encoder 获取特征就判断是否对齐，而 ITM 让图像、文本特征经过多模态层之后再判断是否匹配。

对于 ITM 而言，判断负样本太简单了，如果简单的随机抽取样本作为负样本的话，模型训练效果不太好。作者在一个 batch 里，通过计算特征相似度，寻找一张图片中除它本身对应的文本之外相似度最高的文本作为负样本。这样就能构建一批 hard negatives，从而提升训练难度。

#Pre-train Images	Training tasks	TR (flickr test)	IR (test)	SNLI-VE (test)	NLVR ² (test-P)	VQA (test-dev)
4M	MLM + ITM	93.96	88.55	77.06	77.51	71.40
	ITC + MLM + ITM	96.55	91.69	79.15	79.88	73.29
	ITC + MLM + ITM _{hard}	97.01	92.16	79.77	80.35	73.81
	ITC _{MoD} + MLM + ITM _{hard}	97.33	92.43	79.99	80.34	74.06
	Full (ITC _{MoD} + MLM _{MoD} + ITM _{hard})	97.47	92.58	80.12	80.44	74.42
	ALBEF (Full + MoD _{Downstream})	97.83	92.65	80.30	80.50	74.54
14M	ALBEF	98.70	94.07	80.91	83.14	75.84

Table 1: Evaluation of the proposed methods on four downstream V+L tasks. For text-retrieval (TR) and image-retrieval (IR), we report the average of R@1, R@5 and R@10. ITC: image-text contrastive learning. MLM: masked language modeling. ITM_{hard}: image-text matching with contrastive hard negative mining. MoD: momentum distillation. MoD_{Downstream}: momentum distillation on downstream tasks.

11 BLIP

网上爬下来的图像文本对，大部分都是不干净的

12 CoCa

减少文本特征提取模型 forward 的次数

13 BeiTv3

做大做强

14 SAM

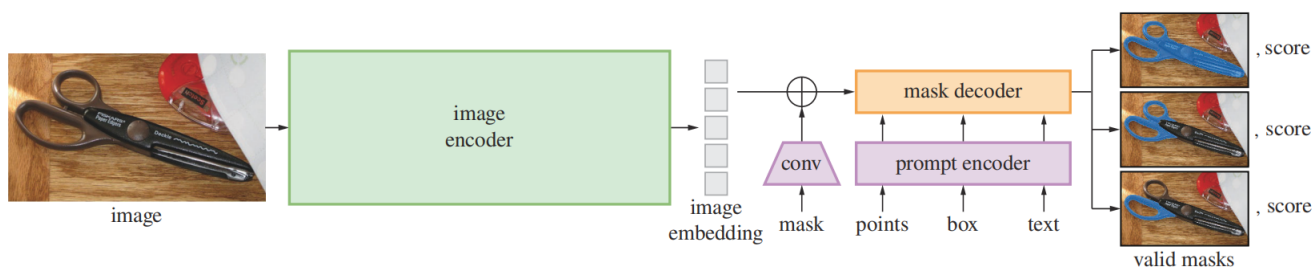


Figure 4: Segment Anything Model (SAM) overview. A heavyweight image encoder outputs an image embedding that can then be efficiently queried by a variety of input prompts to produce object masks at amortized real-time speed. For ambiguous prompts corresponding to more than one object, SAM can output multiple valid masks and associated confidence scores.

提醒： 输入是什么？

图片和 prompt、

14.1 image encoder

利用 mae 预训练的 vit，最低限度适应高分辨率的输入，该 encoder 在 prompt encoder 之前，对每张图像只运行一次。

输入 (c, h, w) 的图像，对图像进行缩放，按照长边缩放成 1024，短边不够就 pad，得到 $(c, 1024, 1024)$ 的图像，经过 image encoder，得到对图像 16 倍下采样的 feature，大小为 $(256, 64, 64)$ 。

14.2 prompt encoder

分成 2 类：稀疏的 (点, box, 文本)，稠密的 (mask)

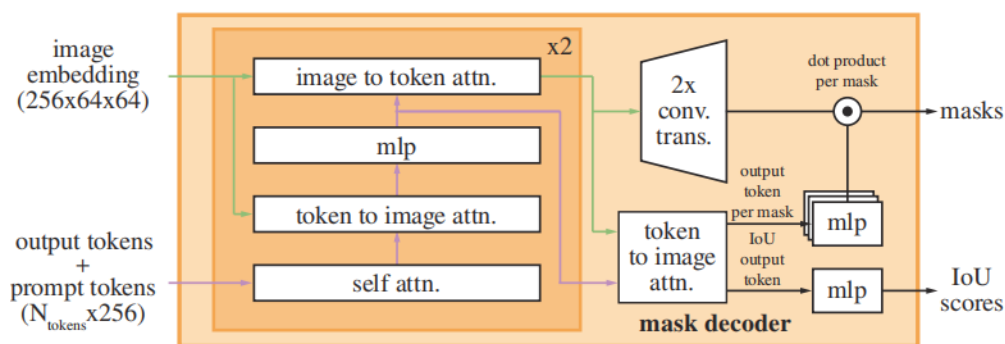
- point: 【点坐标调整加上 0.5 是为了向中心移动】映射到 256 维的向量，【填充处理，为了允许模型处理不同数量的点提示时保持输入一致】包含代表点位置的 positional encoding，加 2 个代表该点是前景/背景的可学习的 embedding

- box: 用一个 embedding 对表示
- 文本: 通过 CLIP 模型进行文本编码
- mask: 用输入图像 1/4 分辨率的 mask, 然后用 (2,2) 卷积核, stride=2 输出【共下采样 16 倍】, 再用 (1,1) 卷积核将 channel 升到 256. mask 和 image embedding 通过 element-wise 相乘【可以理解成 mask 的 feature 对 image 的 feature 进行加权】

映射出的特征的 channel 和 image embedding 的 channel 一致【因为后面要用 attention 进行融合】。

14.3 mask decoder

在 prompt embeddings 中插入一个可学习的 token【iou token 和 mask token】, 用于 decoder 的输出



- prompt tokens + output tokens 进行 self attn
- 用得到的 token 和 image embedding 进行 cross attn (token 作为 Q)
- point-wise MLP 更新 token
- 用 image embedding 和 (3) 的 token 进行 cross atten (image embedding 作为 Q)

重复上述步骤 2 次, 再将 attn 再通过残差进行连接, 最终输出 masks 和 iou scores。

为了解决输出模糊性问题【一个提示可能生成多个 mask】, 预测输出多个 masks, 在训练过程中, 只回传最小的 loss, 为了对 mask 进行排序, 增加一个小的 head 预测 mask 和目标的 iou。

当输入多个提示时, 生成的 mask 会比较接近, 为了减少 loss 退化和确保获取明确的 mask, 此时只预测一个 mask

提醒: masks 的输出维度是 $B \times C \times H \times W$, 其中 B 是输入的 prompt 数量, C 是 mask 的数量, (H, W) 为输入图像尺寸

提醒: 解码器部分的结构设计依据?

15 YOLACT

并行预测当前图片的原型掩码 (prototype mask) 和每个 bbox 实例的掩码系数 (mask coefficient), 然后将原型与掩模系数线性组合来生成实例掩码 (instance masks)

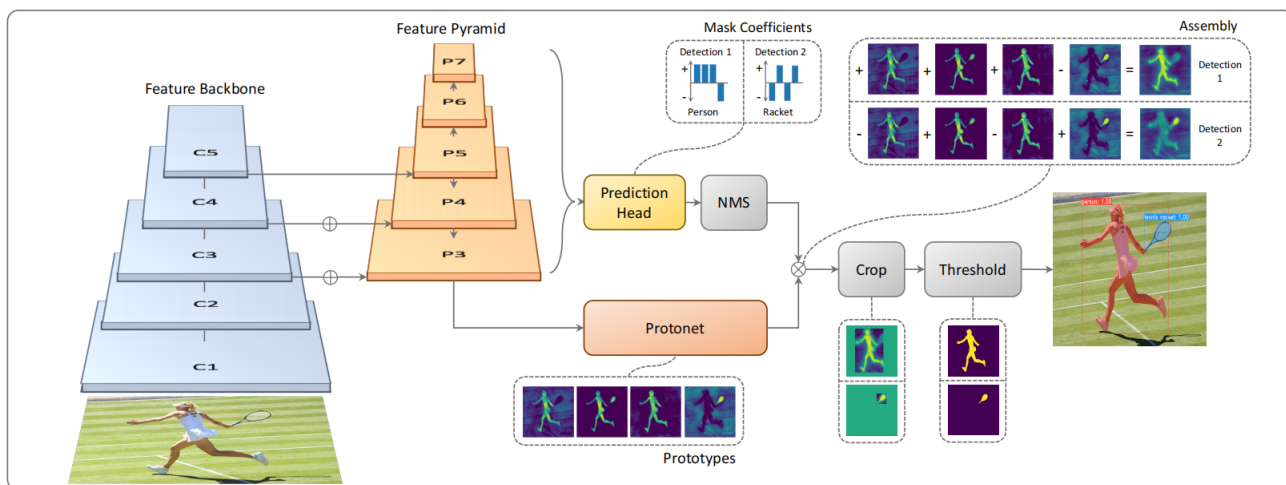


Figure 2: **YOLACT Architecture** Blue/yellow indicates low/high values in the prototypes, gray nodes indicate functions that are not trained, and $k = 4$ in this example. We base this architecture off of RetinaNet [27] using ResNet-101 + FPN.

15.1 训练过程

- bbox 预测分支: 每个 bbox 的 mask 系数没有标签, 故 bbox head 分支仅仅对 bbox $[(N \times 4)]$ 和 cls $[(N \times 81)]$ 分支计算 loss, mask 系数 $[(N \times 32)]$ 监督信息来自后面的原型掩码预测分支
- 掩码预测分支: 不能用预测的 bbox 来裁剪【前期不稳定】, 基于正样本 anchor, 采用 gt bbox 去裁剪对应的预测 mask 图, 就可以得到实例级别 mask【直接采用所有 gt bbox 去 crop, 可能会存在 bbox 分支和 protonet 分支监督点不一致的问题】

```

prototypes = self.protonet(x)
prototypes = prototypes.permute(0, 2, 3, 1).contiguous()

# idx表示图片索引
cur_sampling_results = sampling_results[idx]
# 找出正样本索引
pos_assigned_gt_inds = cur_sampling_results.pos_assigned_gt_inds

# cur_bboxes是gt bbox, (M, 4)
bboxes_for_cropping = cur_bboxes[pos_assigned_gt_inds].clone()
pos_inds = cur_sampling_results.pos_inds
# cur_coeff_pred是正样本索引所对应的mask系数预测值(M, 32)
cur_coeff_pred = cur_coeff_pred[pos_inds]

# 每个实例对应的mask预测图(138, 138, M)
mask_pred = cur_prototypes @ cur_coeff_pred.t()
mask_pred = torch.sigmoid(mask_pred)

```

```
# 基于bbox裁剪出mask图
mask_pred = self.crop(mask_pred, bboxes_for_cropping)

# 后面就可以计算bce loss了
```

15.2 推理过程

- 遍历每个预测输出层，对每个层先利用 `nms_pre` 参数过滤到指定数目的框；对这些框进行解码操作；最后对所有结果采用 `nms` 进行抑制得到指定数目的 `bbox` 和对应类别、`mask` 系数值，默认最多是 100
- 为了使得 `mask` 更加精确，这里只选择了 FPN 后输出的最大尺度特征图预测全局原型，特征图上面每个位置都采用长度为 32 的 `prototypes` 来表征，然后将 `N` 个预测框和 `prototypes` 矩阵进行乘加操作，即可得到每个 `bbox` 对应的 `mask`
- 利用 `bbox` 坐标去 `mask` 图上进行切割即可得到对应的 `mask` 图
- 二值化，插值函数将 `bbox` 和 `mask` 都还原到原始图尺度，最终得到实例分割结果

16 FastSAM

FastSAM 通过两阶段实现，先全景实例分割，再根据提示匹配

16.1 全实例分割

基于 Yolov8-seg 实现，可以参考 YOLACT¹⁵

16.2 目标匹配

- 点提示：通过点提示匹配后，模型会选择与点提示相对应的多个掩码。然后，这些掩码会被合并成一个单一的掩码，以完整地标记感兴趣的对象。在合并过程中，可能会使用形态学操作（如膨胀和腐蚀）来改善掩码的合并效果
- 框提示：每个边界框与分割掩码之间的交并比，选择与用户指定的边界框具有最高 IoU 分数的分割掩码作为最佳匹配
- 文本提示：使用 CLIP 模型的文本编码器来处理文本提示，提取出与文本描述相对应的文本嵌入，再比较文本嵌入与图像特征之间的相似性，以找到与文本描述最匹配的图像区域

17 Grounding DINO

特点：可以根据文字描述检测指定目标
相较于⁸，它的优势是：

- Grounding DINO 的 transformer 结构更接近于 NLP 模型，因此它更容易同时处理图片和文字

- Transformer-based detector 在处理大型数据集时被证明有优势
- 作为 DETR 的变种，DINO 能够完成 end-to-end 的训练，而且不需要 NMS 等额外的后处理

越多越早的特征融合带来的收益越大

在以上结论的指导下，作者设计了不同的特征融合方式分别应用在三个阶段上：

- 设计了结合 self-attention、text-to-image cross-attention、image-to-text attention 的特征 enhancer 应用在 Neck 中
- 设计了一个 language-guided query selection 方法用于初始化 Head 的 query；
- 设计了一个跨模态的 decoder 用于 Head 部分，从而增强 query 的特征表达

Swin Transformer 作为 image backbone，BERT 作为 text backbone

17.1 Feature Extraction and Enhancer

